Vivekanand Education Society's Institute Of Technology

Department Of Information Technology

Pseudonym Resolution and Formal Verification of an Unlinkable Two-Message IoT Authentication Protocol

Chaitanya Kelkar, Jay Kerkar, Yash Kolekar, Chirag Poornamath

Project Guide: Dr. Shanta Sondur

24.4 ms per session at 1,000 devices with a **precomputed table** (paper: 23.68 ms)

44 s per session in the same network with the **on-demand scan**

8 dropped messages lock a counter-based device out **for good**

125 tests pass (96% coverage); session key proven secret for **any** correct server

Introduction

- Lightweight IoT protocols hide a device behind a **pseudonym**, a disguised ID that changes every time period, so an eavesdropper cannot link its sessions.
- Al-Rasheed et al. (IEEE TCE, 2026) propose one: **two messages, hashing and symmetric encryption only**.
- **The catch:** the server can decrypt a message only once it knows who sent it, and the pseudonym hides exactly that.

Problem Statement

- Message M1 shows only the pseudonym; the rest is encrypted with a per-device key.
- So every server must first **find the sender** by recomputing pseudonyms from its secrets: **pseudonym resolution**.
- The specification never defines this step, and its cost model charges a **fixed 23.68 ms** per session for any number of devices.
- Two open choices set the cost: **time periods** (counter or clock) and **search** (scan on demand or precomputed table).

What does resolution really cost, and which choice should a deployment make?

Design Space

	How the server finds the sender	Probes, honest	Forged
O1	Counter + scan	$(N–1)W/2+L+2$	$N·W$
O2	Counter + table	1	0
O3	Clock + scan	$(N–1)W_t/2+1$	$N·W_t$
O4	Clock + table	0 (+N/period)	0
TM	Rainbow time-memory table	$\approx 0.5–0.7\ S^{2/3}$	$\approx 4\ S^{2/3}$
CR	Randomised hash lock *	(N+1)/2	N
MW	Key tree *	$\leq \delta·d$	$\leq \delta$

Probe = one hash (0.32 ms). N devices; W = 25 periods (L = 16 back, F = 8 ahead); $W_t = 3$; $S = 128N$; $\delta = 32$. * Adds a random value to M1.

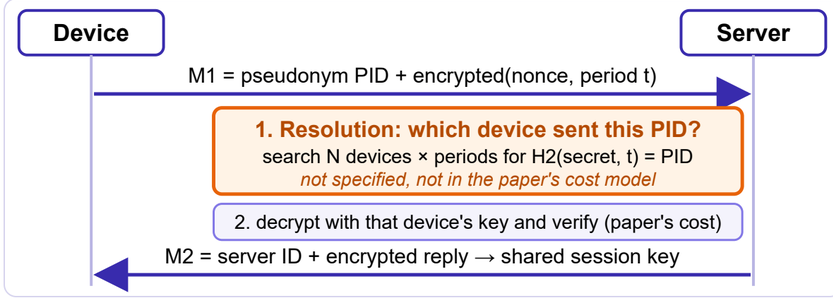
Desynchronisation

- A counter device advances on every attempt; the server looks only **8 periods ahead**.
- Dropping **8 messages in a row locks the device out for good** (7 drops: it recovers). Tested for O1 and O2.
- Clock periods, CR and MW cannot fall out of sync.

Future Scope

- Stateful proof of replay protection (e.g. Tamarin).
- Hardware timings on the ESP8266; full CR and MW messages.

Methodology



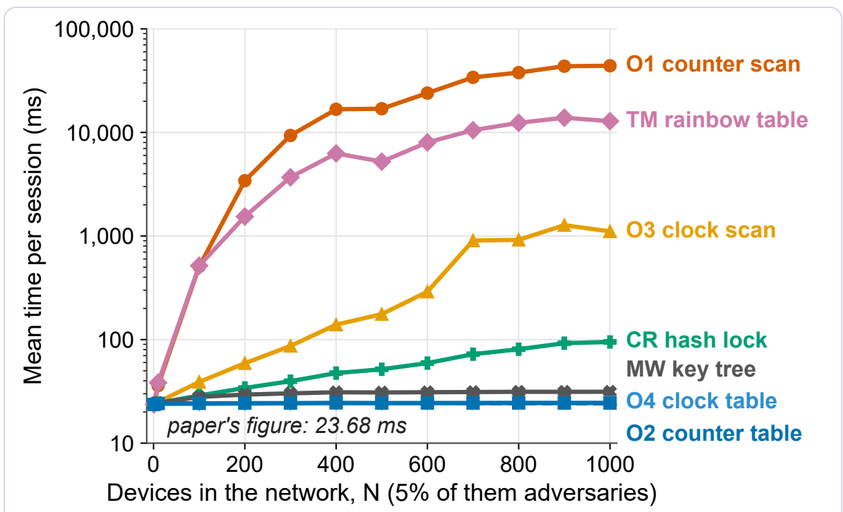
01 Reproduce Python and ESP8266 C versions of the exchange match independent test vectors byte for byte.

02 Count Seven strategies, 1 to 10,000 devices; every hash counted and checked against formulas.

03 Simulate OMNeT++ 6.4 model of the paper's network; 5% adversaries forging 1 msg/s; 960 runs.

04 Prove ProVerif 2.05 models of secrecy, authentication and unlinkability.

Results



Mean session time (log scale), OMNeT++ model of the paper's network (1500×1500 m, 10 Mbps, servers = 5% of devices), five seeds.

#	Strategy	100	500	1,000 devices	KB/server
1	O2 counter + table	24.25	24.40	**24.40 ms**	92.9
2	O4 clock + table	23.93	24.22	**24.59 ms**	46.4
3	MW key tree	27.96	30.86	31.38 ms	10.4
4	CR hash lock	28.86	51.71	94.89 ms	10.1
5	O3 clock + scan	38.99	176.1	1.11 s	16.1
6	TM rainbow table	516.5	5,228	12.9 s	59.9
7	O1 counter + scan	507.9	16,947	**44.0 s**	12.1

Mean session time in ms, ranked at 1,000 devices; all strategies give the paper's 23.68 ms at one device.

- **Tables (O2, O4) stay within 1 ms of the paper** up to 1,000 devices, for 93 KB and 46 KB per server.
- **The counter scan (O1) reaches 44 s**: honest load alone gives 19.2 s, and each forged pseudonym forces a full scan the block-list never sees.
- **A forged message costs O1 25,000 hashes** at 1,000 devices, but O2 and O4 nothing: a table miss needs no hash.

Formal Verification

ProVerif 2.05, active attacker. The server may match a pseudonym to **any** enrolled device, so results hold for every correct resolver.

Property	Result
Q1/Q2 Session key stays secret	✓ holds
Q3 Server knows who it talked to	✓ holds
Q4 … and never accepts M1 twice	✗ replay
Q5 Device and server agree on the key	✓ holds
FS Old keys safe after a key leak	✗ not claimed
U1 Unlinkable, counter periods	✓ holds
U2 Unlinkable, clock periods	✗ same PID

Q4 needs the replay state the specification leaves open; with clock periods, two sessions in one period share a PID and can be linked.

Implementation

- **Python package** (crypto, trusted authority, device, server, five resolvers, TCP demo) and an **ESP8266 C port**: 5 of 5 handshakes agree.
- **125 tests pass**, 96% coverage: unit, property (10,000 trials) and adversarial.

```
$ hea-device authenticate ...
Mutual authentication succeeded with 'server-A'.
  round-trip latency: 2.103 ms
$ hea-device authenticate ... --replay-last
REJECTED by server: ReplayDetected (stale epoch 0)
```

Conclusion

- **Counter periods:** precompute a table (O2), one probe per login, and add a resync path.
- **Clocks:** O4 uses half the memory, but same-period sessions are linkable.
- **Scan on demand** (O1, O3) only in small, trusted networks.
- **The specification** should state these choices and cost resolution.

References

1. A. Al-Rasheed et al., "A low-cost hybrid ECC authentication protocol for trustworthy IoT communication," IEEE Trans. Consum. Electron., 72(2), 4483–4491, 2026.
2. S. A. Weis et al., "Security and privacy aspects of low-cost RFID systems," SPC 2003.
3. M. Ohkubo et al., "Cryptographic approach to 'privacy-friendly' tags," 2003.
4. D. Molnar and D. Wagner, "Privacy and security in library RFID," ACM CCS 2004.
5. G. Avoine et al., "Reducing time complexity in RFID systems," SAC 2005.
6. B. Blanchet, "Modeling and verifying security protocols with ProVerif," FnT Priv. Secur., 2016.

Code and data: github.com/jaykerkar0405/hybrid-ecc-iot-auth

Find the sender first: a precomputed table keeps 24.4 ms at 1,000 devices; an on-demand scan takes 44 s.